Operating Systems Lab 10

Synchronization Using Mutex and Semaphores

Lab Outline

By the end of this lab, students will be able to:

- Understand Mutex and Semaphore synchronization mechanisms.
- Learn how Mutex prevents multiple threads from entering a critical section simultaneously.
- Understand Producer-Consumer synchronization.
- Learn the difference between Mutex and Semaphore.
- Implement synchronization using pthread Mutex and Semaphore libraries.
- Understand the Dining Philosophers problem and Deadlock.
- Learn techniques to avoid deadlock.

Introduction

In the previous lab, we studied synchronization using hardware instructions such as Compare-and-Set. Although hardware solutions are effective, programmers rarely use assembly instructions directly in real-world applications.

Modern operating systems provide synchronization libraries that are easier to use and maintain. Two of the most commonly used synchronization mechanisms are:

- Mutex
- Semaphore

Both are used to protect shared resources and prevent race conditions when multiple threads access the same data simultaneously.

In this lab, we will study how Mutex and Semaphore work and how they are used to solve classical synchronization problems.

Mutex Lock

What is a Mutex?

Mutex stands for **Mutual Exclusion**.

A mutex is a lock that ensures only one thread can access a critical section at a time.

Suppose two threads are modifying the same variable. If both threads enter the critical section simultaneously, incorrect results may occur. A mutex prevents this by allowing only one thread to acquire the lock.

The thread that acquires the lock enters the critical section. Other threads must wait until the lock is released.

Important Mutex Functions

Initialize Mutex

```
pthread_mutex_init(&mutex,NULL);
```

Creates and initializes a mutex.

Lock Mutex

```
pthread_mutex_lock(&mutex);
```

Acquires the lock.

If another thread already owns the lock, the current thread waits.

Unlock Mutex

```
pthread_mutex_unlock(&mutex);
```

Releases the lock so another thread can enter the critical section.

Example 1

Producer-Consumer Problem Using Mutex

Scenario

Suppose a teacher is generating quiz questions and placing them in a shared buffer.

A student reads questions from the same buffer.

The teacher acts as a **Producer** and the student acts as a **Consumer**.

Since both threads access shared variables, synchronization is required.

Program

```
#include<stdio.h>
#include<pthread.h>
#include<unistd.h>

pthread_mutex_t mutex;

int n=10;
int buffer[10];

int in=0, out=0, count=0;

void* producer(void* ptr)
{
    for(unsigned int i=1;i<15;i++)
    {
        while(count==n);

        buffer[in]=i;
        in=(in+1)%n;

        pthread_mutex_lock(&mutex);
        count++;
        pthread_mutex_unlock(&mutex);
    }

    pthread_exit(0);
}

void* consumer(void* ptr)
{
    for(unsigned int i=1;i<15;i++)
    {
        while(count==0);

        printf("%d\n",buffer[out]);

        out=(out+1)%n;

        pthread_mutex_lock(&mutex);
        count--;
        pthread_mutex_unlock(&mutex);
    }

    pthread_exit(0);
```

```
}

int main()
{
    pthread_t pid1,pid2;

    pthread_mutex_init(&mutex,NULL);

    pthread_create(&pid2,NULL,&consumer,NULL);
    pthread_create(&pid1,NULL,&producer,NULL);

    pthread_join(pid1,NULL);
    pthread_join(pid2,NULL);

    return 0;
}
```

Discussion

The producer continuously inserts data into the buffer.

The consumer continuously removes data from the buffer.

The variable:

```
count
```

is shared between both threads.

Without synchronization, both threads may update `count` simultaneously and create incorrect results.

Mutex protects the critical section:

```
pthread_mutex_lock(&mutex);
count++;
pthread_mutex_unlock(&mutex);
```

and

```
pthread_mutex_lock(&mutex);
count--;
pthread_mutex_unlock(&mutex);
```

This guarantees that only one thread modifies `count` at a time.

Semaphore

What is a Semaphore?

A semaphore is a synchronization variable used to control access to shared resources.

Unlike a mutex, a semaphore can hold values greater than one.

Semaphores are mainly of two types:

Binary Semaphore

A binary semaphore can have only two values:

```
0 or 1
```

It behaves similarly to a mutex.

Counting Semaphore

A counting semaphore can store multiple values.

Example:

```
0
1
2
3
...
```

It is useful when multiple instances of a resource are available.

Important Semaphore Functions

Initialize Semaphore

```
sem_init(&sem,0,value);
```

Creates and initializes a semaphore.

Wait Operation

```
sem_wait(&sem);
```

Decrements the semaphore.

If the semaphore value is zero, the thread waits.

Post Operation

```
sem_post(&sem);
```

Increments the semaphore.

If another thread is waiting, it may proceed.

Example 2

Bounded Buffer Problem Using Semaphore

Scenario

Suppose a warehouse has 10 storage spaces.

A producer stores items in the warehouse.

A consumer removes items from the warehouse.

The producer should not insert items when the warehouse is full.

The consumer should not remove items when the warehouse is empty.

Semaphores can efficiently solve this problem.

Program

```
#include<stdio.h>
#include<pthread.h>
#include<unistd.h>
#include<semaphore.h>

sem_t full,empty;

int n=10;
int buffer[10];

int in=0,out=0;
```

```
void* producer(void* ptr)
{
    for(unsigned int i=1;i<8;i++)
    {
        sem_wait(&empty);

        buffer[in]=i;

        in=(in+1)%n;

        sem_post(&full);
    }

    pthread_exit(0);
}

void* consumer(void* ptr)
{
    for(unsigned int i=1;i<8;i++)
    {
        sem_wait(&full);

        printf("%d\n",buffer[out]);

        out=(out+1)%n;

        sem_post(&empty);
    }

    pthread_exit(0);
}

int main()
{
    pthread_t pid1,pid2;

    sem_init(&full,0,0);

    sem_init(&empty,0,10);

    pthread_create(&pid2,NULL,&consumer,NULL);
    pthread_create(&pid1,NULL,&producer,NULL);

    pthread_join(pid1,NULL);
    pthread_join(pid2,NULL);

    return 0;
}
```

Discussion

Two semaphores are used:

```
full
```

Number of occupied slots.

```
empty
```

Number of empty slots.

Initially:

```
full = 0
empty = 10
```

Producer:

```
sem_wait(&empty);
```

Checks whether empty space exists.

If yes, it inserts data.

After inserting data:

```
sem_post(&full);
```

indicating that one more item is available.

Consumer:

```
sem_wait(&full);
```

Checks whether data is available.

After consuming data:

```
sem_post(&empty);
```

indicating that one more slot is available.

This prevents both overflow and underflow of the buffer.

Mutex vs Semaphore

Mutex	Semaphore
Used for mutual exclusion	Used for synchronization and resource management
Binary lock	Counter-based mechanism
Ownership exists	No ownership

Mutex	Semaphore
One thread at a time	Can manage multiple resources
Simpler	More flexible

Dining Philosophers Problem

Scenario

Five philosophers are sitting around a circular table.

Between every two philosophers, there is one chopstick.

Each philosopher needs two chopsticks to eat:

- Left Chopstick
- Right Chopstick

The philosopher alternates between:

```
Thinking
Eating
Thinking
Eating
```

Basic Solution

```
wait(chopstick[i]);
wait(chopstick[(i+1)%5]);

/* eat */

signal(chopstick[i]);
signal(chopstick[(i+1)%5]);

/* think */
```

Problem with Basic Solution

Suppose all philosophers become hungry at the same time.

Each philosopher picks up the right chopstick.

Now:

```
Philosopher 1 waits for left chopstick
Philosopher 2 waits for left chopstick
Philosopher 3 waits for left chopstick
Philosopher 4 waits for left chopstick
Philosopher 5 waits for left chopstick
```

Nobody can proceed.

The system becomes permanently blocked.

This situation is called:

Deadlock

Deadlock-Free Solution 1

Allow Only Four Philosophers

Instead of allowing all five philosophers to sit simultaneously:

```
Maximum philosophers allowed = 4
```

At least one philosopher remains outside.

This breaks the circular waiting condition.

Deadlock cannot occur.

Deadlock-Free Solution 2

Pick Both Chopsticks Together

A philosopher may pick up chopsticks only if:

```
Left chopstick available
AND
Right chopstick available
```

Otherwise, the philosopher waits.

This prevents partial resource allocation.

Deadlock is avoided.

Deadlock-Free Solution 3

Asymmetric Solution

Odd philosophers:

```
Left first
Right second
```

Even philosophers:

```
Right first
Left second
```

Since all philosophers do not follow the same order, circular waiting is eliminated.

Deadlock is avoided.

Graded Task 1

Objective

Implement all three deadlock-free solutions separately.

Create:

- Solution 1 File
- Solution 2 File
- Solution 3 File

Each file should demonstrate one deadlock prevention technique.

Graded Task 2

Objective

Deadlock prevention alone is not enough.

We also want maximum concurrency.

If all philosophers arrive together:

```
Two philosophers can eat simultaneously.
```

Your task is to design a solution that:

- Avoids deadlock
- Allows the maximum possible number of philosophers to eat at the same time

Practice Question 1

Scenario

A printing server receives print jobs from multiple users.

Create two threads:

- Thread 1 adds print jobs.
- Thread 2 processes print jobs.

Use a Mutex lock to synchronize access to the shared job counter.

Practice Question 2

Scenario

A parking area contains 20 parking spaces.

Cars enter and leave continuously.

Use a counting semaphore to manage available parking slots and ensure that the number of cars never exceeds capacity.

Conclusion

Mutex and Semaphores are practical synchronization tools provided by operating systems.

Mutex is mainly used to protect critical sections by allowing only one thread at a time.

Semaphores provide a more flexible mechanism for coordinating multiple threads and managing shared resources.

These concepts are widely used in real-world operating systems, multithreaded applications, databases, web servers, and concurrent software systems.